Coding Assignment — Observability for a Multi-Agent System

Estimated time: 20 minutes **Format:** Individual, hands-on coding **Runs fully offline** — no cloud account, AWS credentials, network access, or external packages required (Python standard library only).

> **Lineage of this exercise.** It's adapted from the AWS pattern in Tracking a multipart upload with the AWS SDKs, where a *progress listener* fires repeatedly as a large job advances and the sample stops at printing raw progress. A multi-agent run has the identical shape — one job decomposed across many workers — and the same gap between "a print statement" and "an observable system." Here you transplant that listener pattern onto agents.

Scenario

A client runs an agentic workflow: an orchestrator hands a task down a pipeline of agents (Planner → Researcher → Writer → Reviewer). When a run misbehaves overnight — it stalls, loops, or silently produces garbage — the team has almost nothing to debug with. The orchestrator just prints a stream of "agent X did step Y," with no way to tell *which* agent was slow, *where* a run died, or *how far* it got.

Your job is to take that minimal listener and build a small **observability layer** that an operations team could actually use to answer those questions.

Learning objective

Multi-agent systems are hard to observe precisely because work is distributed, nondeterministic, and easy to lose track of across handoffs. The skill being exercised: turning raw per-agent progress signals into **correlated, structured, queryable telemetry** — events you could feed into a trace viewer or log pipeline and actually reason about.

Setup

- Python 3.9+. No `pip install`, no credentials, no network.
- Save the starter code below as `agents.py` and run `python agents.py`.

Starter code (the "before" state — minimal observability)

```
python
```

```python
import random
import time

class Agent:
    """A simulated agent that does N steps of work. Pure simulation — no LLM, n
    def __init__(self, name, steps, fail_at_step=None):
        self.name = name
        self.steps = steps
        self.fail_at_step = fail_at_step  # set to an int to force a determinis

    def run(self, listener):
        for step in range(1, self.steps + 1):
            time.sleep(random.uniform(0.05, 0.2))  # simulate work / latency
            if self.fail_at_step and step == self.fail_at_step:
                raise RuntimeError(f"{self.name} failed at step {step}")
            listener(self.name, step, self.steps)


class Orchestrator:
    def __init__(self, agents, listener):
        self.agents = agents
        self.listener = listener

    def run(self):
        for agent in self.agents:
            agent.run(self.listener)


def progress_listener(agent_name, step, total_steps):
    # The "before" state: an unstructured print, just like the AWS sample's byt
    print(f"{agent_name}: step {step}/{total_steps}")


def main():
    agents = [
        Agent("Planner", 3),
        Agent("Researcher", 6),
        Agent("Writer", 4),
        Agent("Reviewer", 2),
        # To exercise the failure path, set e.g.  Agent("Writer", 4, fail_at_st
    ]
    Orchestrator(agents, progress_listener).run()
    print("Done")
```

```
if __name__ == "__main__":
    main()
```

Run it once. That wall of `agent: step X/Y` lines is everything the team currently has. Now make it observable.

Tasks

Work in order; each builds on the previous. Time-boxes are guidance.

1. Correlate every event (≈4 min)

A line that says "Researcher: step 3" is useless across concurrent or repeated runs. Give each **run** a `trace_id` and each **agent execution** a `span_id` (use `uuid`), and stamp both onto every event so you can group all events from one run, and all events from one agent within it.

2. Progress and pace (≈5 min)

The listener only knows one agent's local step. Compute pipeline-level signals:

- **Pipeline percent complete** — steps finished across *all* agents ÷ total steps.
- **Throughput** — steps per second since the run started.
- Per-agent **duration** when an agent finishes.

3. Structured, lifecycle-based events (≈6 min)

Replace `print(...)` with **structured JSON** events emitted on meaningful transitions, not on every tick:

- `agent_started` and `agent_completed` for each agent.
- `agent_progress` **throttled** to roughly every 25% of an agent's steps (agent traces get noisy fast — over-emitting is its own failure mode).

Each event should carry at minimum: `timestamp`, `trace_id`, `span_id`, `event`, `agent`, and the relevant progress/pace fields.

4. Failure localization + run summary (≈5 min)

Wrap the orchestration so that:

- On an agent raising, you emit `agent_failed` capturing **which** agent, **which step**, the error message, and the **pipeline percent complete at the moment it died** — then stop the run cleanly.
- At the end (success *or* failure) you emit one `run_summary`: final status, total duration, agents completed, and which agent failed (if any).

Test it by setting a `fail_at_step` on one agent and confirming the failure event pinpoints exactly where the run broke.

Stretch goal (only if time remains)

Pick one:

- **Persist a trace.** Append every event to a `trace.jsonl` file (no credentials needed) and write a tiny function that reads it back and prints a per-agent timeline.
- **Detect a stall.** Flag any agent whose gap between steps exceeds a threshold and emit an `agent_stalled` warning event.
- **Fan-out.** Let one agent spawn child sub-agents and verify your `trace_id`/`span_id` model still lets you reconstruct the parent/child tree.

Acceptance criteria

Your finished program should:

1. Emit **structured JSON events**, not free-text prints.
2. **Correlate** events via a run-level `trace_id` and per-agent `span_id`.
3. Report **pipeline percent complete, throughput, and per-agent duration**.
4. **Throttle** progress events instead of emitting one per step.
5. Emit a **failure event that localizes** which agent/step broke and how far the run got, plus a final **run summary**.
6. Run end-to-end with `python agents.py` — no setup beyond the standard library.

Example of a target event

```json
{"timestamp": "2026-06-24T09:14:02Z", "trace_id": "5f2c…", "span_id": "a91b…",
```

Discussion (cover briefly if reviewing as a group)

- What makes observability *harder* for multi-agent systems than for a linear job — nondeterminism, loops, fan-out, agents calling agents? Which of these does your `trace_id`/`span_id` model already handle, and which would break it?
- The `trace_id` / `span_id` scheme here is a simplified version of distributed tracing (e.g. OpenTelemetry spans). Where would these events go in a real deployment, and what would you add to make per-agent spans nest correctly?
- "Step X/Y" is a poor health signal. Beyond progress, what *agent-specific* signals matter in production — token/cost per agent, tool-call success rate, loop/retry counts, output quality checks — and which would you instrument first?

Submission

Submit your `agents.py` plus a 2–3 line note: which extra signal you'd capture next for a real agentic workload, and where you'd send these events in a client environment.